Linear RNNs

are all you need?

SIAN SHINJO

https://github.com/noss123/linear-rnns

Modelling Sequences

Basics

Let's start with the simplest "*translation*" task: given a set of datapoints $\{(x_1, y_1), \dots, (x_N, y_N)\}$, model the mapping f from input vectors $x \in \mathbb{R}^N_{in}$ to output vectors $y \in \mathbb{R}^N_{out}$.

For the simplest case, this can be accomplished by a ***multilayer perceptron*** **(MLP)**. This is a series of stacked layers of the form $h^{(k+1)} = \phi\left(W_k h^{(k)}\right)$, with $h^{(0)} = x$ being the input and $h^{(L+1)} = \hat{y}$ being the output, assuming there are L hidden layers.

This is possible due to the ***universality property*** which states that for any unknown function f there exists parameters $W_1, \dots, W_L$ such that the MLP approximates arbitrarily close the target function.

But this is only applicable to vectors of **fixed dimension**. Real-world translation tasks involve sequences of *varying lengths*, with input and output sequences not necessarily having the same number of tokens.

Modelling Sequences

Sequences of varying length

A more general formulation is modelling the mapping f from input vectors $x \in \mathbb{R}_{in}^{T}$ to output vectors $y \in \mathbb{R}_{out}^{T'}$ where $T, T' \in [0, \infty)$, such that we may have $T \neq T'$.

This is equivalent to learning an arbitrary program, and is solved by adding a ***non-linear recurrence*** to the perceptron. The system we end up with is as follows:

$$h_{t+1} = \phi\left(W_r h_t + W_i x_t\right)$$
$$y_t = W_o h_t$$

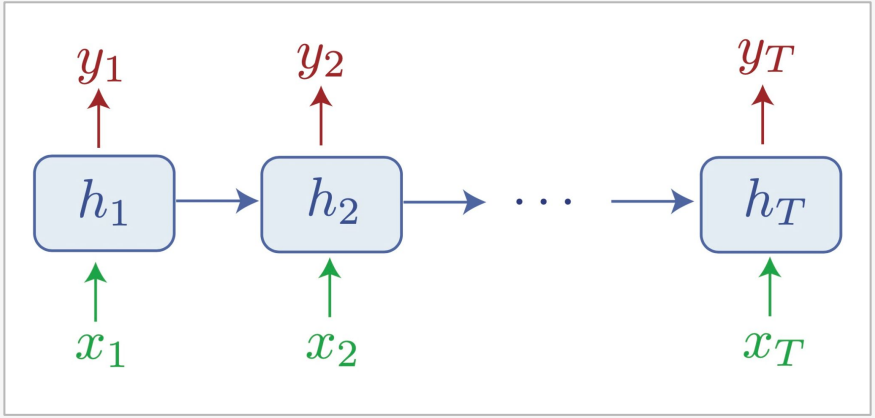
But this is the expression of a **regular (*vanilla*) RNN**!

This formulation possesses a ***dynamical universality property*** which allows it to approximate arbitrarily close any *non-linear dynamical system* $y_{t+1} = f(y_t, x_t)$.

RNNs: A Refresher

Or, a guide to speedy inference

Maintains a history of previous inputs in the form of a hidden state h_k , which is updated via a *non-linear recurrence*.



RNN architecture

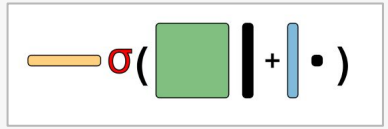
★ Theoretically ***infinite context window***, practically suffers from *vanishing or exploding gradients*; finite hidden state vector is an information bottleneck.

★ **Training:** $\mathcal{O}(T)$ for the entire sequence, ***not parallelisable***.

★ **Inference:** $\mathcal{O}(1)$ for each token, which means $\mathcal{O}(T)$ for the entire sequence.

$$h_k = \sigma\left(\overline{A}h_{k-1} + \overline{B}x_k\right)$$
$$y_k = \overline{C}h_k$$

Update step



Transformers: A Refresher

Attention, but at what cost?

- Each token attends to every other token.
- Since there is no temporal dependency (unlike RNNs), the training can be massively parallelised.
- Inference still happens autoregressively with quadratic time complexity.

★ Has a ***finite context window***, limited by quadratically increasing memory and use of positional encodings to understand sequence order.

★ **Training:** $\mathcal{O}(T^2)$ for the entire sequence, ***highly parallelisable.***

★ **Inference:** *Slow*, with $\mathcal{O}(T)$ for each token (with KV-caching enabled), so $\mathcal{O}(T^2)$ for the entire sequence.

Linear RNNs?

A middle ground

Some questions: *The RNN formulation naturally speeds up inference costs, so is there a way to parallelise training? How do we deal with the vanishing/exploding gradients problem?*

- Vanilla RNNs are prone to vanishing/exploding gradients due to the *non-linearity applied* at each hidden state update, and also the *nature of eigenvalues* in the recurrence.
- Eigenvalues of A above 1 are ***exploding directions***, while those close to 0 are ***fast decaying directions***; hence we must constrain the recurrence matrix A to avoid both.
- If we remove the non-linearity, we can also parallelise training.

*But can we just remove the non-linearity like this? Won't it affect the **expressivity** of the model?*

$$
\begin{aligned}
h_t &= \mathbf{A}h_{t-1} + \mathbf{B}x_t \\
z_t &= \mathbf{C}h_t + Dx_t \\
y_t &= \hat{f}_{\text{out}}(z_t)
\end{aligned}
$$

Linear RNNs?

Eigenvalue behaviour

By themselves, linear RNNs have limited expressivity. They exhibit a fixed point at 0, and depending on the nature of eigenvalues of the recurrence matrix, they show activity that either:

- diverges to infinity ($|\lambda| > 1$)
- converges (decays) to fixed point at zero ($|\lambda| < 1$)
- stays idle, i.e. line attractor ($\lambda = 1$)
- oscillates in concentric circles (pair of complex λ_i's with $|\lambda| = 1$ and $\lambda \neq 1$)

To avoid divergence to infinity, we **restrict $|\lambda| < 1$**; this inevitably leads to some decay over time. This corresponds to forgetting information in the hidden state with time. Since we want to preserve information over long contexts, we try to **keep $|\lambda|$ close to 1** for the λ_i's.

This by itself will not allow the modified RNN to perform context-dependent operations (e.g. figuring out that "not bad" is positive, and not the valences of "not" and "bad" summed together).

So we need to introduce the non-linearity back in somehow, but how do we do that?

Linear RNNs?

Tokenwise non-linearity!

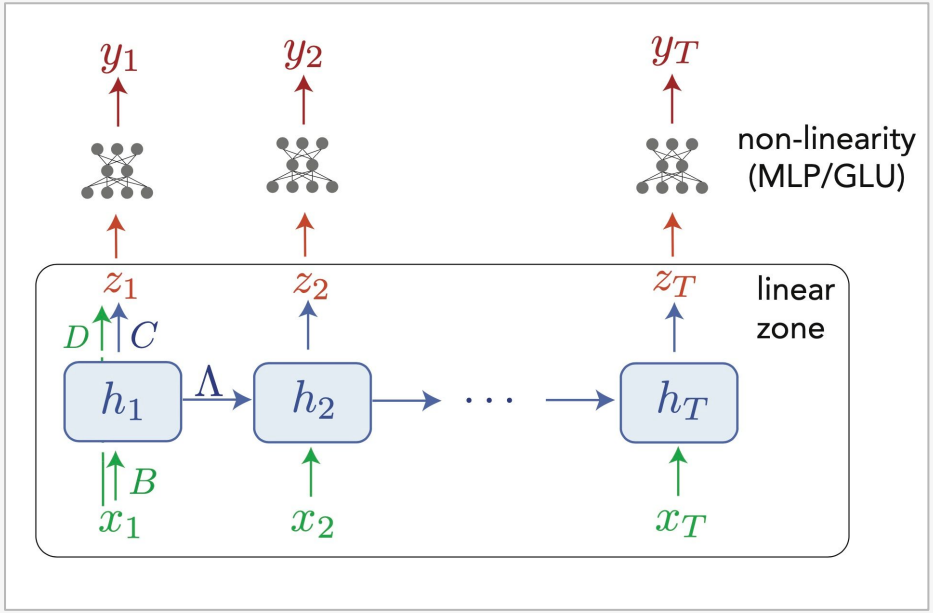
$$
\begin{aligned}
h_t &= \mathbf{A}h_{t-1} + \mathbf{B}x_t \\
z_t &= \mathbf{C}h_t + Dx_t \\
y_t &= \hat{f}_{\text{out}}(z_t)
\end{aligned}
$$

LRNN formulation

We add a non-linear mapping $\hat{f}_{\text{out}}$ from the hidden states h_t to the final outputs y_t.

This can be done via a simple MLP or GLU applied over the linear readout of hidden state Ch_t (with skip connection Dx_t added without modifying representational capacity).

Since ***all non-linearities are applied over single tokens***, they do not occur in computations impacting the time dynamics of internal states! This allows us to proceed with parallelising training.



LRNN design, as per linear recurrent unit (LRU) model introduced by Orvieto et al. (2023) [4]

[4] Orvieto et al., "Resurrecting Recurrent Neural Networks for Long Sequences," arXiv:2303.06349, 2023

Linear RNNs?

Enter the convolution

The linear form allows us to rewrite the recurrent representation as a ***discrete convolution*** by *unrolling* [2]. We can see this by expanding:

$$h_0 = \overline{\mathbf{B}}x_0 \qquad h_1 = \overline{\mathbf{AB}}x_0 + \overline{\mathbf{B}}x_1 \qquad h_2 = \overline{\mathbf{A}}^2\overline{\mathbf{B}}x_0 + \overline{\mathbf{AB}}x_1 + \overline{\mathbf{B}}x_2 \qquad \dots$$

$$y_0 = \overline{\mathbf{C}}\overline{\mathbf{B}}x_0 \qquad y_1 = \overline{\mathbf{C}}\overline{\mathbf{AB}}x_0 + \overline{\mathbf{C}}\overline{\mathbf{B}}x_1 \qquad y_2 = \overline{\mathbf{C}}\overline{\mathbf{A}}^2\overline{\mathbf{B}}x_0 + \overline{\mathbf{C}}\overline{\mathbf{AB}}x_1 + \overline{\mathbf{C}}\overline{\mathbf{B}}x_2 \qquad \dots$$

$$y_k = \overline{\mathbf{C}}\overline{\mathbf{A}}^k\overline{\mathbf{B}}x_0 + \overline{\mathbf{C}}\overline{\mathbf{A}}^{k-1}\overline{\mathbf{B}}x_1 + \cdots + \overline{\mathbf{C}}\overline{\mathbf{AB}}x_{k-1} + \overline{\mathbf{C}}\overline{\mathbf{B}}x_k$$

$$y = \overline{\mathbf{K}} * x$$

$$\overline{\mathbf{K}} \in \mathbb{R}^L = (\overline{\mathbf{C}}\overline{\mathbf{B}}, \overline{\mathbf{C}}\overline{\mathbf{AB}}, \dots, \overline{\mathbf{C}}\overline{\mathbf{A}}^{L-1}\overline{\mathbf{B}})$$

[2] Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396, 2021

Linear RNNs?

Parallelising training via FFT (convolution in Fourier space)

The convolutional form is possible due to the ***time-invariance*** of A, B, C matrices, due to which they remain constant over all tokens. Computing the convolution directly in the time domain is $\mathcal{O}(L^2)$.

By the Convolution Theorem, a ***convolution in the time domain is equivalent to pointwise multiplication in the frequency domain***.

$$y = \overline{K} * x = \mathrm{iFFT}\left(\mathrm{FFT}\left(\overline{K}\right) \odot \mathrm{FFT}\left(x\right)\right)$$

1. *Zero-pad both the kernel K and input sequence x to a length of $2L$ to prevent circular wrap-around errors.*
2. *Apply the Fast Fourier Transform (FFT) to both padded vectors, an operation that runs in $\mathcal{O}(L \log L)$ time.*
3. *Perform pointwise multiplication on the transformed frequency vectors, taking $\mathcal{O}(L)$ time.*
4. *Apply the inverse FFT (iFFT) to convert the product back to the time domain, which costs $\mathcal{O}(L)$, and truncate the sequence to keep only the first L entries.*

We can thus train as a convolution with $\mathcal{O}(L \log L)$ complexity, and infer as a recurrence in linear time.

[2] Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396, 2021

State Space Models (S4)

A continuous-time latent state model

An SSM maps a 1-D input signal $u(t)$ to an N-D latent state $x(t)$ before projecting to a 1-D output signal $y(t)$.

$$x'(t) = \boldsymbol{A}x(t) + \boldsymbol{B}u(t)$$
$$y(t) = \boldsymbol{C}x(t) + \boldsymbol{D}u(t)$$

Here, matrices **A, B, C, D** are ***time-invariant*** (fixed for all *t*) parameters learned by *gradient descent*. Since D can be modelled as a skip connection, it can be ignored here. The model has linear dynamics and is *differentiable* end-to-end.

The basic SSM model has poor performance, since linear first-order ODEs solve to an exponential function due to which gradients may scale exponentially in the sequence length. So, ***careful initialisation of matrix A is required***.

[2] Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396, 2021

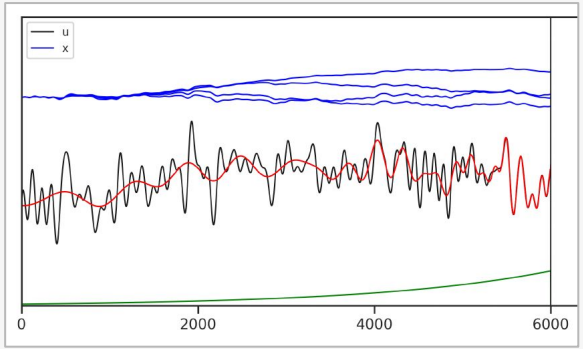
HiPPO Matrix

Addressing long-range dependencies

We build the matrix A such that the hidden state $h(t)$ summarises the history of inputs $x(t)$. The hidden state stores a ***vector of coefficients*** that compresses the input seen so far by projecting it onto the ***orthogonal polynomial basis of Legendre polynomials***.

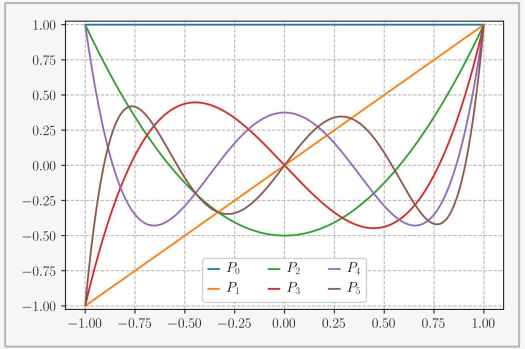
This is similar to how a Fourier transformation decomposes a signal into a summation of sinusoidal functions that closely approximates the original signal.

The main difference here is that the ***recent signal is more precise***, while older portion is decayed exponentially. So, the hidden state captures more information about tokens seen recently as compared to older tokens.



(HiPPO Matrix)

$$A_{nk} = -\begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}.$$



[1] Gu et al., "HiPPO: Recurrent Memory with Optimal Polynomial Projections," arXiv:2008.07669, 2020; [2] Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396, 2021

Discretisation

From continuous signal to recurrence

Since our input tokens form a discrete sequence $(u_0, u_1, ...)$ instead of a continuous function $u(t)$, we must discretise the continuous-time SSM model using a step size Δ that represents the ***resolution of input***.

This is conceptually equivalent to sampling the inputs u_k from an implicit continuous signal $u(t)$, where $u_k = u(k\Delta)$.

$$x_k = \overline{\boldsymbol{A}} x_{k-1} + \overline{\boldsymbol{B}} u_k \qquad \overline{\boldsymbol{A}} = (\boldsymbol{I} - \Delta/2 \cdot \boldsymbol{A})^{-1}(\boldsymbol{I} + \Delta/2 \cdot \boldsymbol{A})$$
$$y_k = \overline{\boldsymbol{C}} x_k \qquad \overline{\boldsymbol{B}} = (\boldsymbol{I} - \Delta/2 \cdot \boldsymbol{A})^{-1} \Delta \boldsymbol{B} \qquad \overline{\boldsymbol{C}} = \boldsymbol{C}.$$

Bilinear method for discretisation used in S4 [2]

Zero-Order Hold (ZOH) discretisation used in Mamba [5]

$$\overline{\boldsymbol{A}} = \exp(\Delta \boldsymbol{A}) \qquad \overline{\boldsymbol{B}} = (\Delta \boldsymbol{A})^{-1}(\exp(\Delta \boldsymbol{A}) - \boldsymbol{I}) \cdot \Delta \boldsymbol{B}$$

[2] Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396, 2021; [5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Diagonalisation

Linear algebra ftw

The recurrence matrix A is a dense N x N matrix, so computing the convolution kernel is expensive. We can improve on this by diagonalising the matrix A via orthogonal diagonalisation.

$$A = V \Lambda V^{-1} \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_N)$$

The matrices V and V^{-1} are absorbed into matrices B and C without loss of generality.

$$h_t = \sum_{k=0}^{t} \Lambda^k B x_{t-k} + \Lambda^t h_0$$

Since the matrix A has been diagonalised, computing Λ^k simplifies to exponentiating the eigenvalues λ_i . This reduces computational complexity.

[3] Gupta & Berant, "Diagonal State Spaces Are as Effective as Structured State Spaces," arXiv:2203.14343, 2022

Linear Recurrent Unit (LRU)

Exponential parametrisation?

The LRU scrapped the continuous-time derivations of SSM models and showed that a discrete-time diagonal linear RNN can match SSM performance if parametrised correctly.

$$h_t = \Lambda h_{t-1} + \gamma \odot (B x_t)$$
$$y_t = \mathrm{Re}(C h_t) + D x_t$$

LRU formulation

magnitude phase

$$\lambda_j = \exp(-\exp(\nu_j^{\log}) + i \exp(\theta_j^{\log}))$$

Stable exponential parametrization

$$\gamma_j \leftarrow \left(1 - |\lambda_j|^2\right)^{1/2}$$

Normalization

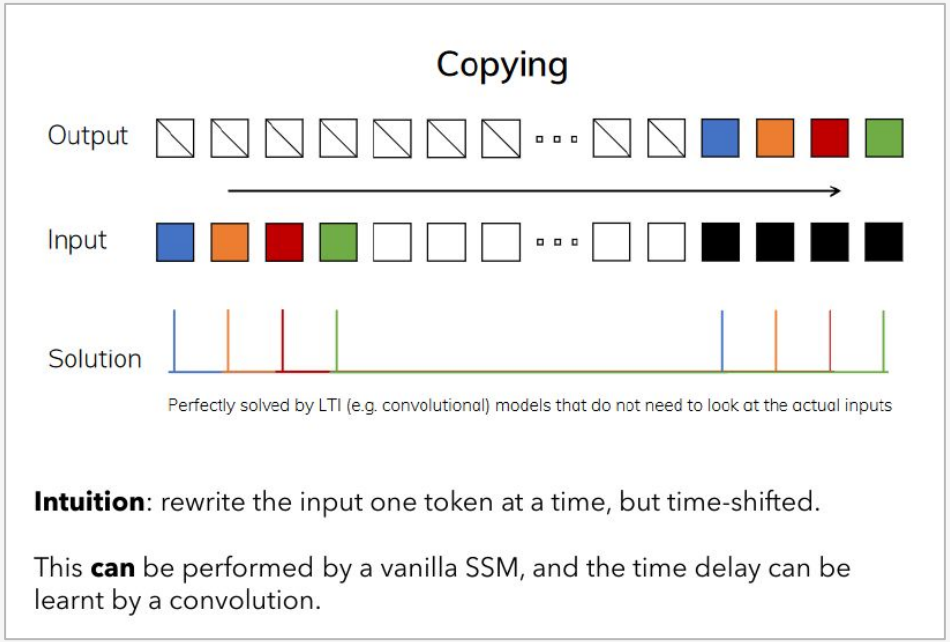
The matrices B and C are complex. The exponential parametrisation guarantees that the eigenvalues lie in the range (0, 1), so that the recurrence is stable by construction. The angle θ_j gives the oscillation frequency of the hidden state. The γ_j is used for normalisation, to prevent unstable activations in the forward pass.

[4] Orvieto et al., "Resurrecting Recurrent Neural Networks for Long Sequences," arXiv:2303.06349, 2023

The Copying Task

Who said copying is easy?

The popular copying task is easily performed by SSMs, which can learn time delay using a convolution.

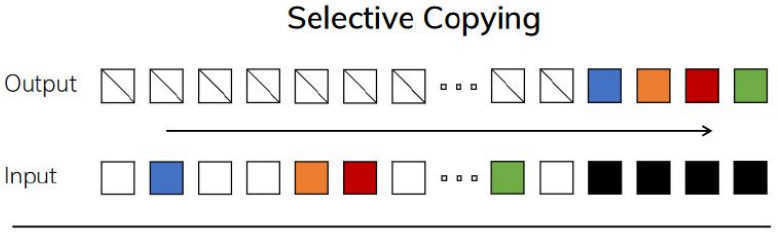


[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Where SSMs Fail

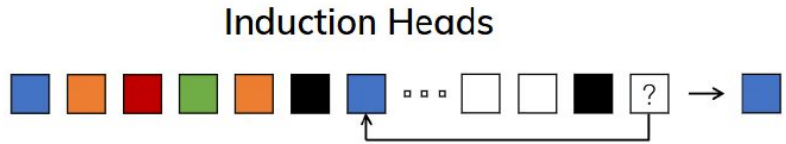
Time-invariant assumption isn't valid

Due to ***time-invariance***, the A, B, C matrices remain the same for each token, causing these failures.



Intuition: given a comment on Twitter, rewrite the comment by removing all the bad words (the white tokens).

This **cannot** be performed by a vanilla SSM, because it requires content-aware reasoning, which vanilla SSM cannot do because they are time invariant (which means the parameters A, B, C, D are the same for every token it generates).



Intuition: for example with "few-shot" prompting we can "teach" LLMs new tasks and how to perform them. With Transformer-based model, this task can be done "easily", because Transformer-based models can attend to the previous tokens when generating the current one, so they can "recall previous history".

This task **cannot** be performed by a time-invariant SSM, because they cannot "select" which previous token to recall from their history.

Failure cases

[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Where SSMs Fail

Time-invariant assumption isn't valid

- The Selective Copying task modifies the popular Copying task (Arjovsky, Shah, and Bengio 2016) by varying the position of the tokens to memorize. It requires content-aware reasoning to be able to memorize the relevant tokens (colored) and filter out the irrelevant ones (white).
- The Induction Heads task is a well-known mechanism hypothesized to explain the majority of in-context learning abilities of LLMs (Olsson et al. 2022). It requires context-aware reasoning to know when to produce the correct output in the appropriate context (black).

These tasks reveal the failure mode of LTI models. From the recurrent view, their constant dynamics (e.g. the $(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}})$ transitions in (2)) cannot let them select the correct information from their context, or affect the hidden state passed along the sequence an in input-dependent way. From the convolutional view, it is known that global convolutions can solve the vanilla Copying task (Romero et al. 2021) because it only requires time-awareness, but that they have difficulty with the Selective Copying task because of lack of content-awareness (Figure 2). More concretely, the spacing between inputs-to-outputs is varying and cannot be modeled by static convolution kernels.

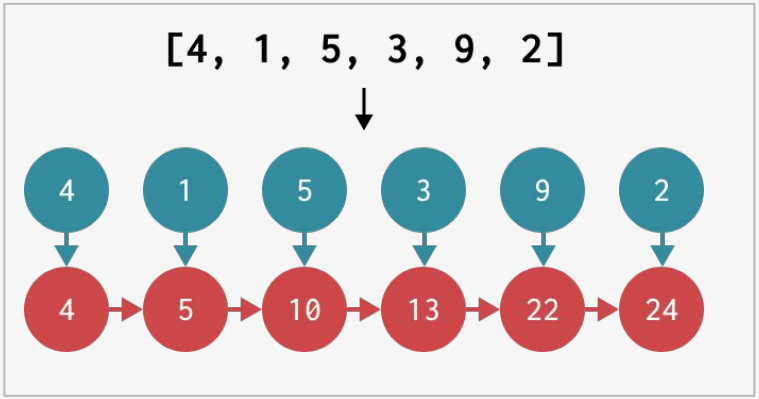
We need to introduce ***time-varying*** nature to allow ***selectivity***; but this means we can no longer use a convolution to parallelise training. Only the recurrence form of the model remains usable.

[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

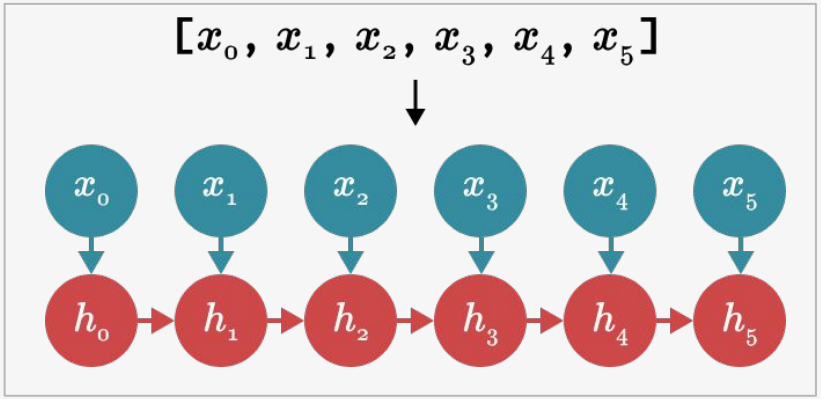
Mamba: The Parallel (Associative) Scan

Another way to parallelise?

Inspired by ***prefix-sum arrays*** used in competitive programming, the parallel scan algorithm provides a technique to compute the sequential formulation quickly in parallel.



Prefix-sum array building



RNN formulation

[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

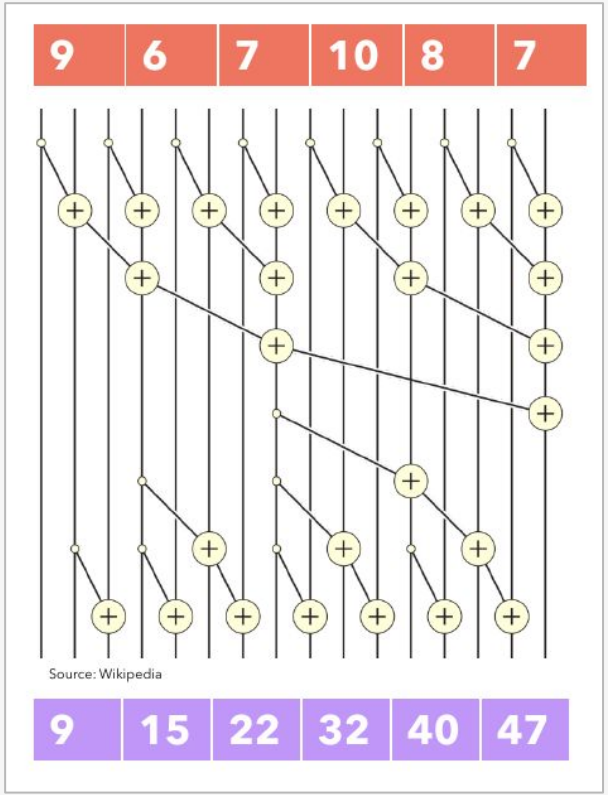
Mamba: The Parallel Scan

Another way to parallelise?

As long as the operations involved are ***associative*** in nature, the order in which operations are done does not matter.

We spawn multiple threads to perform the binary operation in parallel, synchronising at each step.

The time complexity is reduced from $\mathcal{O}(N)$ to $\mathcal{O}(N/T)$, where N is the number of tokens in the sequence and T is the number of parallel threads.



[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

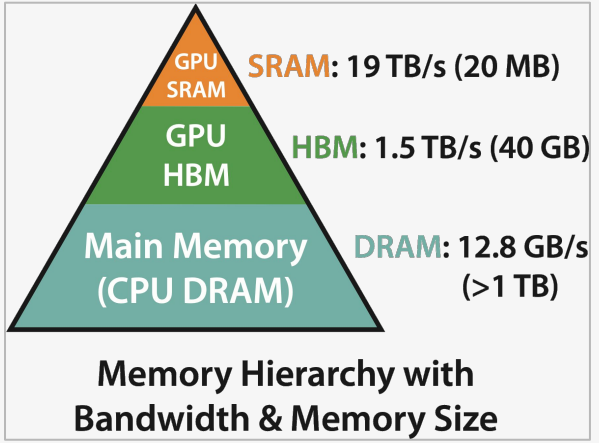
Mamba: Kernel Fusion and Recomputation

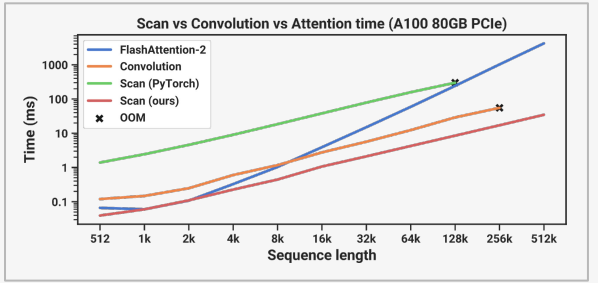
Other hardware fixes

The parallel scan algorithm as written in PyTorch initially performed worse than FlashAttention-2. This was found to be due to the ***memory-bound*** nature of the operations — the speed of transfer between the HBM and SRAM of the GPU was the actual bottleneck.

While GPU computation speed itself was very high (several TFLOPS/s), the saving and loading of intermediate results to the HBM was occupying most of the operation time. This was solved by ***fusing individual kernels into a custom CUDA kernel***. This allowed all operations to be performed sequentially without saving intermediate states.

The caching and resume of activations during backpropagation was also scrapped in favour of ***faster recomputation*** for the same reason.





[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Mamba: Parameters

Adding time-varying structure

Parameters modified to be a function of the current token, so now B, C and Δ vary with time.

Algorithm 1 SSM (S4)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
3: $\boldsymbol{C} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
4: $\Delta : (\mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter})$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_B(x)$
3: $\boldsymbol{C} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_C(x)$
4: $\Delta : (\mathsf{B}, \mathsf{L}, \mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{B}, \mathsf{L}, \mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

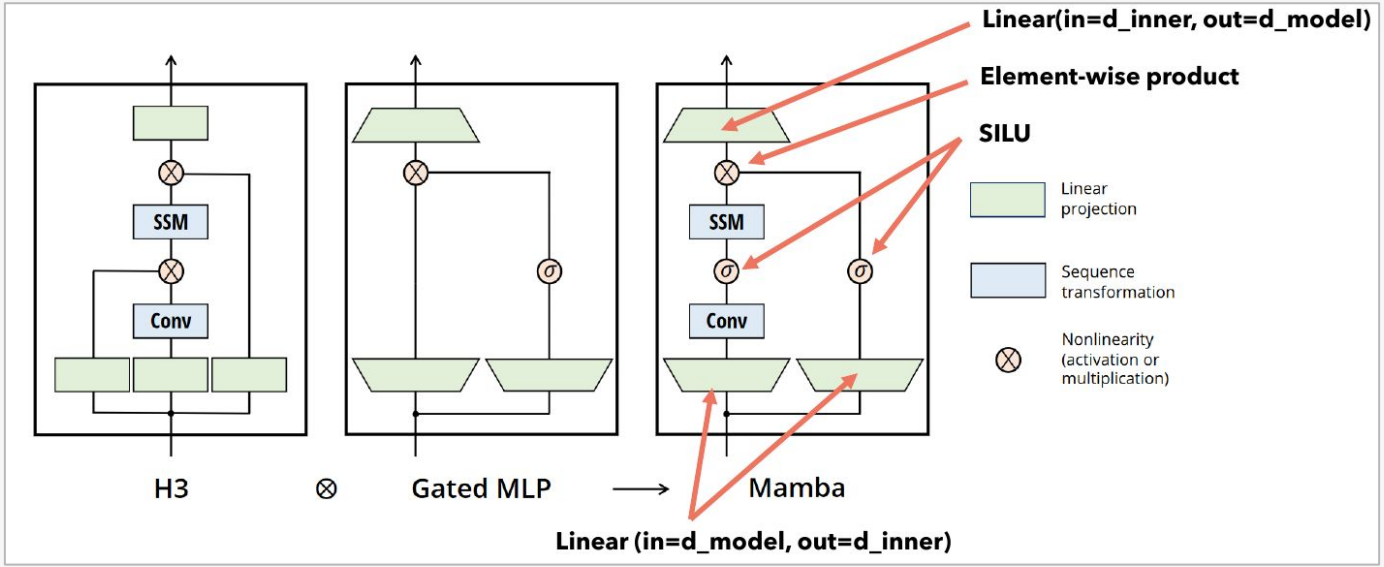
Comparing the two algorithms

[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Mamba: Architecture

Or, how to build a snake

Combination of Gated MLP and SSM dynamically filters history according to current token.



The Mamba block, which is stacked in layers, similar to a Transformer

[5] Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023

Mamba-2

Even faster training?!

Guiding questions:

- What makes Transformers so fast is exploiting the tensor cores of GPUs through matrix multiplications; the hardware is highly optimised to compute matmuls in parallel. Can we exploit this somehow?
- Are SSMs and attention completely unrelated?

What we need to do:

- Modify the SSM formulation so that most of the computations occur via ***matmuls***
- Allow ***larger state***, which improves in-context memory capacity (works on the information bottleneck).

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Wait, it's analogous to attention?!

The diagonal recurrence matrix A is replaced with a ***scalar times identity*** structure. So, its shape changes from (T, N) to simply (T). This *sacrifices expressivity for increased efficiency* due to parallelisation.

$$A_t = a_t \cdot I$$

Since we can represent A_t by a scalar now, the recurrence simplifies after unrolling:

$$h_t = a_t h_{t-1} + B_t x_t$$
$$y_t = C_t^\top h_t$$

$$\mathbf{y}_t = \sum_{s \le t} (a_t \dots a_{s+1}) \cdot (C_t^\top B_s) \cdot x_s$$

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Wait, it's analogous to attention?!

Then, let's define the following matrix

$$M = L \circ CB^\top \in \mathbb{R}^{(\mathtt{T},\mathtt{T})} \tag{3}$$

Finally, M encodes a *sequence transformation* $x \in \mathbb{R}^{\mathtt{T}} \to y \in \mathbb{R}^{\mathtt{T}}$ mapping a 1D input to a 1D output—just as in equation (1)—through basic matrix multiplication $y = Mx$.

$$L = \begin{bmatrix} 1 & & & \\ a_1 & 1 & & \\ a_2 a_1 & a_2 & 1 & \\ \vdots & \vdots & \ddots & \ddots \\ a_{\mathtt{T}-1} \dots a_1 & a_{\mathtt{T}-1} \dots a_2 & \dots & a_{\mathtt{T}-1} \quad 1 \end{bmatrix}.$$

What's special about this? Well, you may notice that it looks very similar to an attention computation. In fact, if all $a_t = 1$, then L is simply the lower-triangular *causal mask* and (3) is equivalent to **causal linear attention** [23]:

$$Y = (L \circ QK^\top)V$$

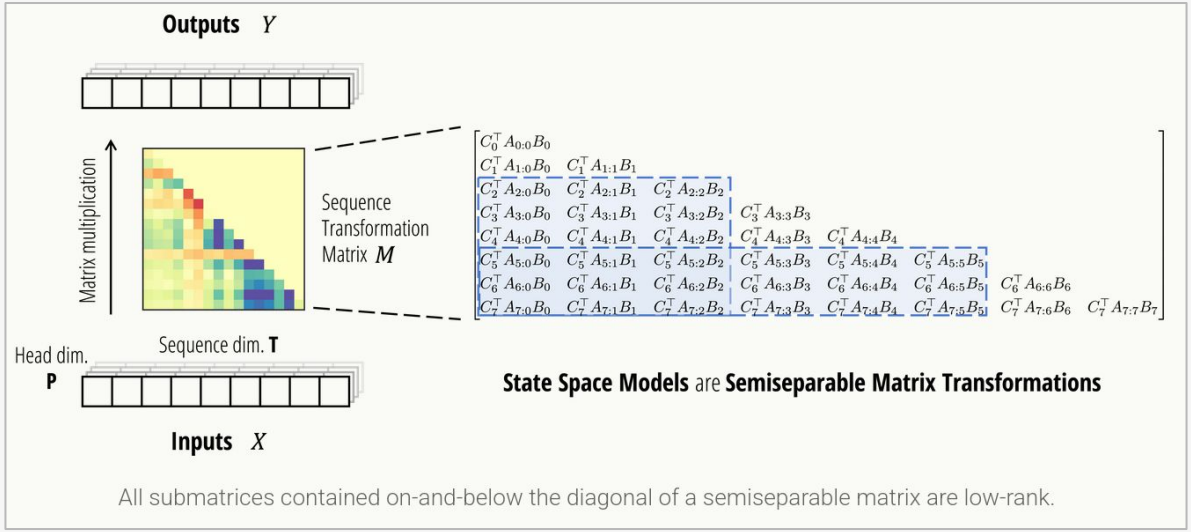
This is exactly the same as equation (3) if we rename $(C, B, X) \mapsto (Q, K, V)$!

How SSD reveals the connection between SSMs and attention in Transformers

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Wait, it's analogous to attention?!



M is a semiseparable matrix, with low rank (<=N) blocks below the diagonal; the specific L chosen makes SSD effectively analogous to linear attention with an input-dependent decay mask

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Block matrix decomposition: the best of both worlds?

We first partition the semiseparable matrix **M** into blocks of size **Q×Q** , and split the sequence into *chunks* of size Q. Then, we use the properties of semiseparable matrices to factorize each off-diagonal block, which is low-rank.

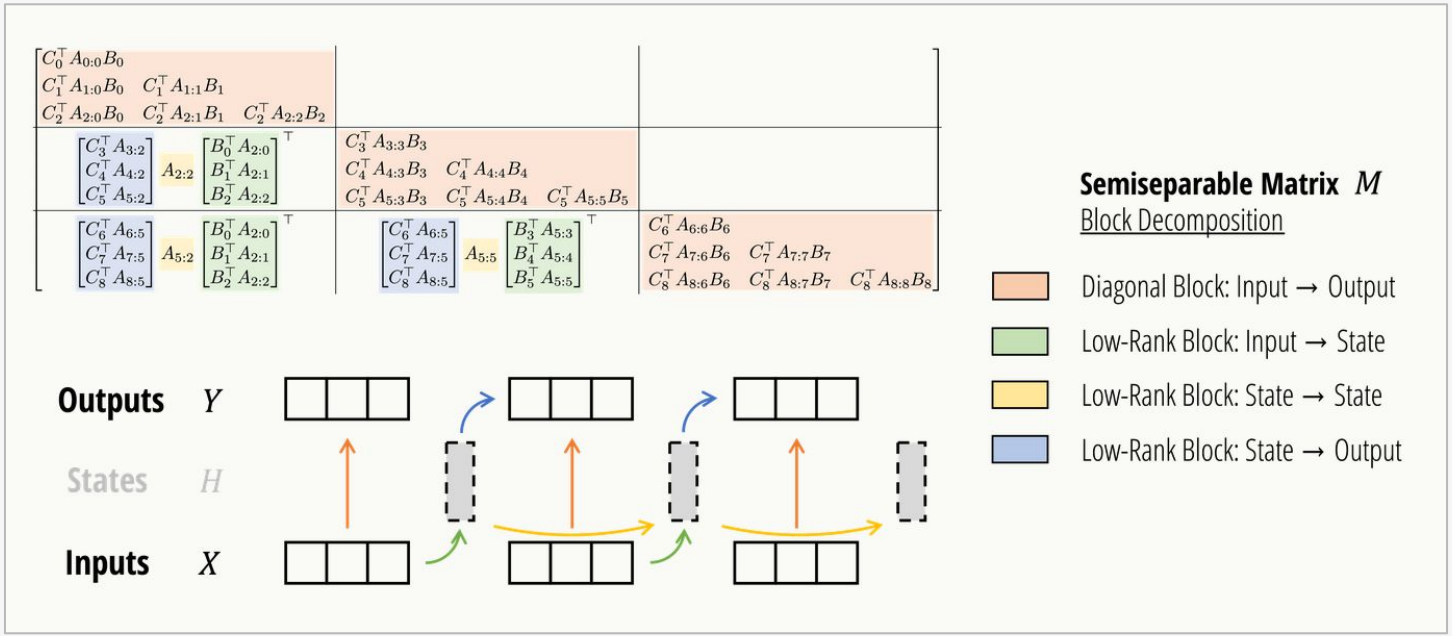
1. **Diagonal Blocks (within a chunk):** Computed using the quadratic, attention-like form. Highly efficient since the chunks are small and perfectly suited for matmuls.
2. **Off-diagonal Blocks (across chunks):** These blocks are low-rank and are factored into three steps:
   a. ***Chunk states:*** Each chunk is compressed into an N-D state using matmuls.
   b. ***State passing:*** A highly efficient scan is run over the T/Q chunk states, where T is the sequence dimension.
   c. ***State to output:*** The contribution of each chunk is read out to the final sequence using matmuls, again.

Since most operations are matmuls, tensor cores are leveraged; 2-8 times faster than Mamba-1.

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Block matrix decomposition: the best of both worlds?



The block matrix decomposition algorithm

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Mamba-2: State-Space Duality (SSD)

Notes about architecture

- **Parallel Projections:** The variables x, B, C, and Δ are generated from a single input projection at the start of the block (similar to Q, K, V in attention), rather than sequentially inside the SSM. This enables tensor parallelism.
- **Multi-head Structure:** The architecture introduces a multi-head design (head dim P = 64) with one scalar a_t per head. B and C are shared across all heads, which is mathematically analogous to multi-value attention (i.e. shared Q/K, per-head V).
- **Extra Normalisation:** To maintain stability at massive scale, *RMSNorm* or *Group Norm* is added after the gating mechanism and before the final output projection.
- **Increased Capacity:** The state size N is drastically increased to 64–256 (up from 16 in Mamba-1).
- **Scope for Hybrid Architectures:** The attention-like design makes it highly compatible for combining with standard Transformer attention layers.

[6] Dao & Gu, "Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," arXiv:2405.21060, 2024

Other Advances

Alternative techniques and recent developments

- **LIGER (Transformer Linearisation):** Provides a mathematical framework to convert existing, fully trained Transformers into constant-memory inference RNNs. By repurposing the Transformer's redundant Key matrices to construct recurrent gating mechanisms, it bypasses the massive compute costs of pretraining Linear RNNs from scratch.
- **Mamba-3 (still preprint):** Interleaves hardware-optimised linear state spaces with periodic global attention layers, recovering maximum reasoning capacity while keeping context windows heavily extended.
- **RWKV-7 (Dynamic State Evolution):** Pushes pure-RNNs past their theoretical ceilings. By making state transitions dynamically dependent on the data itself, it gives parallelizable RNNs the ability to track the kind of complex, non-commutative sequences that previously required classical, unrollable RNNs.

[7] Peng et al., "RWKV-7: Dynamic State Evolution for Pure Recurrent Neural Networks," Preprint, 2025; [8] Zhu et al., "LIGER: Linearizing Pre-trained Transformers via Gated Recurrent Conversions," ICML 2025; [9] Dao & Gu, "Mamba-3: Interleaving Selective State Spaces and Attention for Massive Scale," Preprint, 2026

Resources

Blogs, videos, etc. (all the fun ways to learn)

- ★ 'RNNs strike back' [neurosopher | blog] https://adrian-valente.github.io/2023/10/03/linear-rnns.html
- ★ 'The Annotated S4' [ICLR | blog] https://iclr-blog-track.github.io/2022/03/25/annotated-s4/
- ★ 'Mamba: The Easy Way' [jackcook | blog] https://jackcook.com/2024/02/23/mamba.html
- ★ 'SSD (Mamba-2): The Model' [goombalab | blog] https://goombalab.github.io/blog/2024/mamba2-part1-model/
- ★ 'SSD (Mamba-2): The Theory' [goombalab | blog] https://goombalab.github.io/blog/2024/mamba2-part2-theory/
- ★ 'SSD (Mamba-2): The Algorithm' [goombalab | blog] https://goombalab.github.io/blog/2024/mamba2-part3-algorithm/
- ★ 'SSD (Mamba-2): The Systems' [goombalab | blog] https://goombalab.github.io/blog/2024/mamba2-part4-systems/
- ★ 'Mamba and S4 Explained: Architecture, Parallel Scan, Kernel Fusion, Recurrent, Convolution, Math' [Umar Jamil | video] https://www.youtube.com/watch?v=8Q_tqwpTpVU
- ★ 'Do we need Attention? - Linear RNNs and State Space Models (SSMs) for NLP' [Sasha Rush | video] https://www.youtube.com/watch?v=dKJEpOtVgXc

References

Papers, papers, papers

[1] Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. "*HiPPO: Recurrent Memory with Optimal Polynomial Projections.*" NeurIPS 2020. arXiv:2008.07669

[2] Gu, A., Goel, K., & Ré, C. "*Efficiently Modeling Long Sequences with Structured State Spaces.*" ICLR 2022. arXiv:2111.00396

[3] Gupta, A., & Berant, J. "*Diagonal State Spaces Are as Effective as Structured State Spaces.*" NeurIPS 2022. arXiv:2203.14343

[4] Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. "*Resurrecting Recurrent Neural Networks for Long Sequences.*" ICML 2023. arXiv:2303.06349

[5] Gu, A., & Dao, T. "*Mamba: Linear-Time Sequence Modeling with Selective State Spaces.*" arXiv:2312.00752, 2023

[6] Dao, T., & Gu, A. "*Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality.*" ICML 2024. arXiv:2405.21060

[7] Peng et al., "RWKV-7: Dynamic State Evolution for Pure Recurrent Neural Networks," Preprint, 2025

[8] Zhu et al., "LIGER: Linearizing Pre-trained Transformers via Gated Recurrent Conversions," ICML 2025

[9] Dao & Gu, "Mamba-3: Interleaving Selective State Spaces and Attention for Massive Scale," Preprint, 2026